

LAB 2 · 클록 분주

작성일 2026. 09. 13. · 이해리

빈 템플릿 → 소스 작성 → Icarus 시뮬레이션 → Vivado 구현

목차

프로젝트 시작

RTL·TB·XDC 작성

VS Code 실행과 파형

실험 전 레포트

Vivado 구현

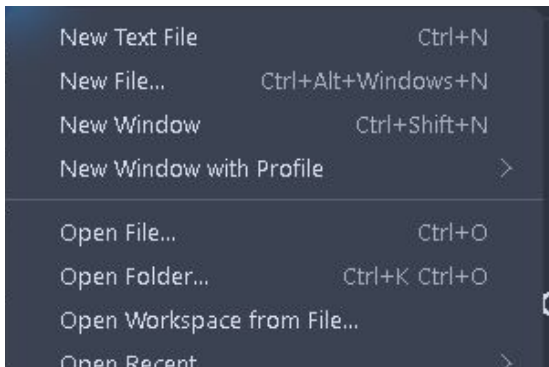
실험 후 레포트

v2.0.1 템플릿을 새 이름으로 clone

Windows PowerShell에서 한 줄씩 실행한다.

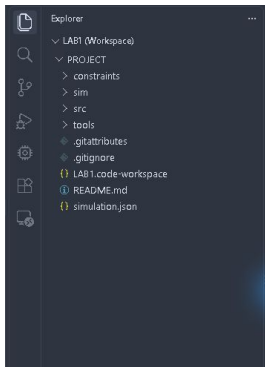
```
git clone --branch v2.0.1 `
https://github.com/Glaysia/fpga-lab-template.git `
lab2_02_clock_divider
cd lab2_02_clock_divider
git switch -c main
code LAB1.code-workspace
```

새 창과 워크스페이스



Ctrl+Shift+N으로 새 창을 연다. File → Open Workspace from File... → 자신의 실습 폴더 → LAB1.code-workspace.

빈 템플릿 확인



clone 직후에는 정답 코드가 없다. Explorer의 `src·sim·constraints`를 펼쳐 빈 파일을 확인하고 뒤의 내용을 작성한다.

workspace와 추천 확장

1. lab2_02_clock_divider 폴더에서 LAB1.code-workspace 선택 → Open.
2. Explorer에 PROJECT 하나와 src.sim.constraints.tools가 보이는지 확인.
3. Extensions → @recommended를 검색.
4. slang.VaporView.vscode-pdf의 Install을 누른다.
5. Python과 Icarus 실행 파일은 별도로 설치되어 있어야 한다.

v2.0.1 편집기와 시뮬레이터

1. slang 확장·서버는 0.3.0을 기준으로 확인한다. 추천 목록은 버전을 고정하지 않는다.
2. 템플릿의 .slang/server.json은 src.sim 탐색용이다. 새 RTL은 simulation.json에도 직접 등록한다.
3. Ctrl+Shift+P → slang: Show Output. 서버 오류가 있으면 README의 버전 확인·재시작 절차를 따른다.
4. slang.path에 다른 사람의 설치 경로를 복사하지 않는다. 직접 지정할 때는 User 설정에 자신의 경로를 쓴다.
5. VS Code 02 Simulate는 Icarus, Vivado Run Behavioral Simulation은 XSim이다.

템플릿 도구 설치·오류 해결 안내

시뮬레이션 도구 확인

1. Terminal → Run Task... → 01 Check tools.
2. Git·Python·iverilog·vvp의 버전이 출력되는지 확인한다.
3. 도구를 찾지 못하면 설치 경로를 PATH에 추가하고 VS Code를 다시 연다.

```
git --version  
python --version  
iverilog -V  
vvp -V
```

빈 파일에서 시작

1. src/design.v를 우클릭 → Rename. 첫 RTL 파일명으로 바꾼다.
2. 추가 RTL은 src를 우클릭 → New File로 만든다.
3. sim/tb_design.v를 이번 TB 파일명으로 바꾼다.
4. constraints/pins.xdc를 이번 XDC 파일명으로 바꾼다.
5. 뒤의 코드 화면을 보며 작성하고 File → Save All.

이번 프로젝트에 작성할 파일

역할	경로
src	src/clock_divider.v
src	src/input_frontend.v
src	src/lab2_clock_divider.v
sim	sim/tb_clock_divider.sv
constraints	constraints/lab2_clock_divider.xdc

simulation.json의 세 항목

`sources`: 앞 표의 `src/` 파일 경로를 배열로 넣는다.

`testbench`: `sim/tb_clock_divider.sv`

`simulation_top`: `tb_clock_divider`

큰따옴표·대괄호·심표를 확인한다. `module` 이름에는 파일 확장자를 붙이지 않는다.

simulation.json · 1-9행

simulation.json를 열고 해당 구간을 직접 입력한다.



```
1 {  
2   "sources": [  
3     "src/clock_divider.v",  
4     "src/input_frontend.v",  
5     "src/lab2_clock_divider.v"  
6   ],  
7   "testbench": "sim/tb_clock_divider.sv",  
8   "simulation_top": "tb_clock_divider"  
9 }
```

두 출력을 구분한다

divided: 입력 주파수를 DIVISOR로 나눈 사각파. 짝수 DIVISOR에서 High와 Low 시간이 같다.

tick: count가 DIVISOR-1일 때만 1인 한 클록 폭의 신호. 다른 순차회로의 enable로 사용할 수 있다.

이 예제는 divided를 다른 always 블록의 클록으로 사용하지 않는다. 모든 상태는 주 클록의 상승 에지에서 바뀐다.

10분주를 손으로 계산

에지 뒤	count	divided	tick
0-4		0	0
5-8		1	0
9		1	1
다음 에지:	0	0	0

오른쪽 항은 에지 직전 count를 읽는다. count==4인 에지에서 divided가 1, count==9인 에지에서 0이 된다.

보드와 테스트벤치의 시간

항목	테스트벤치	실제 보드
입력 주기	10 ns	1 ms
10분주 주기	100 ns	10 ms
1000분주 주기	이 TB에서는 사용 안 함	1 s

TB의 100 MHz는 빠른 기능 검사 조건이다. XDC와 보드의 주 클록은 1 kHz다.

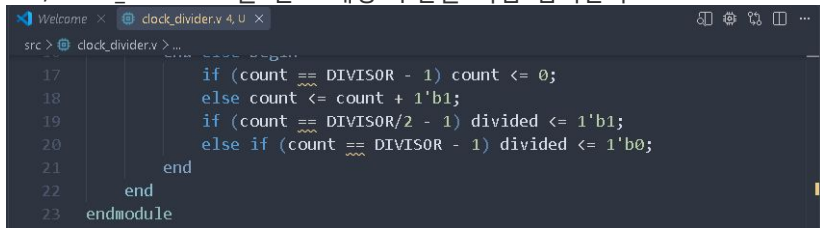
clock_divider.v · 1-16행

src/clock_divider.v를 열고 해당 구간을 직접 입력한다.

```
Welcome x clock_divider.v 4. U x
src > @ clock_divider.v > ...
1 | timescale 1ns/1ps
2 // DIVISOR must be an even integer >= 2.
3 // divided is an observation output; use tick as an enable on clk.
4 module clock_divider #(parameter integer DIVISOR = 10) (
5     input wire clk, rst,
6     output reg divided,
7     output wire tick
8 );
9     localparam integer WIDTH = $clog2(DIVISOR);
10    reg [WIDTH-1:0] count;
11    assign tick = !rst && (count == DIVISOR - 1);
12    always @(posedge clk) begin
13        if (rst) begin
14            count <= 0;
15            divided <= 1'b0;
16        end else begin
```

clock_divider.v · 17-23행

src/clock_divider.v를 열고 해당 구간을 직접 입력한다.



```
src > @ clock_divider.v > ...  
17         if (count == DIVISOR - 1) count <= 0;  
18         else count <= count + 1'b1;  
19         if (count == DIVISOR/2 - 1) divided <= 1'b1;  
20         else if (count == DIVISOR - 1) divided <= 1'b0;  
21     end  
22 end  
23 endmodule
```

input_frontend.v · 1-16행

src/input_frontend.v를 열고 해당 구간을 직접 입력한다.

```
1  timescale 1ns/1ps
2  // Main clock is 1 kHz: STABLE_CYCLES=20 means 20 ms.
3  // Set switches first, then press the button; the bus is not an atomic CDC.
4  module input_frontend #(parameter integer STABLE_CYCLES = 20) (
5      input wire clk, rst, button,
6      input wire [7:0] sw,
7      output wire reset,
8      output reg press,
9      output wire [7:0] switches
10 );
11     (* ASYNC_REG = "TRUE" *) reg [1:0] reset_pipe;
12     (* ASYNC_REG = "TRUE" *) reg button_meta, button_sync;
13     (* ASYNC_REG = "TRUE" *) reg [7:0] sw_meta, sw_sync;
14     localparam integer WIDTH = (STABLE_CYCLES < 2) ? 1 : $clog2(STABLE_CYCLES);
15     reg [WIDTH-1:0] count;
16     reg accepted;
```

input_frontend.v · 17-32행

src/input_frontend.v를 열고 해당 구간을 직접 입력한다.

```
17 assign reset = reset_pipe[1];
18 assign switches = sw_sync;
19 // Asynchronous assertion; release follows two main-clock edges.
20 always @(posedge clk or posedge rst) begin
21     if (rst) reset_pipe <= 2'b11;
22     else reset_pipe <= {reset_pipe[0], 1'b0};
23 end
24 always @(posedge clk or posedge reset) begin
25     if (reset) begin
26         button_meta <= 0; button_sync <= 0;
27         sw_meta <= 0; sw_sync <= 0;
28     end else begin
29         button_meta <= button; button_sync <= button_meta;
30         sw_meta <= sw; sw_sync <= sw_meta;
31     end
32 end
```

input_frontend.v · 33–46행

src/input_frontend.v를 열고 해당 구간을 직접 입력한다.

```
33     always @(posedge clk or posedge reset) begin
34         if (reset) begin
35             count <= 0; accepted <= 0; press <= 0;
36         end else begin
37             press <= 0;
38             if (button_sync == accepted) count <= 0;
39             else if (count == STABLE_CYCLES - 1) begin
40                 accepted <= button_sync;
41                 count <= 0;
42                 press <= button_sync;
43             end else count <= count + 1'b1;
44         end
45     end
46 endmodule
```

lab2_clock_divider.v · 1-16행

src/lab2_clock_divider.v를 열고 해당 구간을 직접 입력한다.

```
clock_divider.v x lab2_clock_divider.v S, U x input_frontend.v 1, U x
src > lab2_clock_divider.v > ...
1 | timescale 1ns/1ps
2 // Combo II-DLD S75. Main clock B6 MUST be set to 1 kHz.
3 // K4: reset. N8: step button. DIPSW1..8 = sw[7]..sw[0].
4 module lab2_clock_divider #(parameter integer STABLE_CYCLES = 20, DIVISOR = 1000) (
5     input wire clk, rst, button,
6     input wire [7:0] sw,
7     output wire [7:0] led
8 );
9     wire reset, press;
10    wire [7:0] switches;
11    input_frontend #(.STABLE_CYCLES(STABLE_CYCLES)) inputs(
12        clk, rst, button, sw, reset, press, switches);
13    wire divided, tick, div2, div10, div50;
14    clock_divider #(.DIVISOR(2)) ratio2(.clk(clk), .rst(reset), .divided(div2), .tick());
15    clock_divider #(.DIVISOR(10)) ratio10(.clk(clk), .rst(reset), .divided(div10), .tick());
16    clock_divider #(.DIVISOR(50)) ratio50(.clk(clk), .rst(reset), .divided(div50), .tick());
```

lab2_clock_divider.v · 17-19행

src/lab2_clock_divider.v를 열고 해당 구간을 직접 입력한다.

```
clock_divider.v U × lab2_clock_divider.v 5, U × input_frontend.v 1, U ×
src > lab2_clock_divider.v >...
17 clock_divider #(.DIVISOR(DIVISOR)) core(clk, reset, divided, tick);
18 assign led = {3'b000, tick, divided, div50, div10, div2};
19 endmodule
```

보드에서 관찰할 출력

출력	주파수 / 폭	관찰 방법
LED[0]	500 Hz	파형·측정 장비
LED[1]	100 Hz	파형·측정 장비
LED[2]	20 Hz	주기 비교
LED[3]	1 Hz	0.5초 켜짐/0.5초 꺼짐
LED[4]	1 ms tick	파형·측정 장비

K4는 초기화 입력이다. 이 회로에서 N8 버튼과 DIP 스위치는 분주비를 바꾸지 않는다. LED[7:5]는 0이다.

테스트벤치가 검사하는 것

1. DIVISOR=10과 최소값 2를 동시에 검사한다.
2. 30개 클록에서 듀티·위상·tick을 매번 비교한다.
3. 10분주 tick을 소비한 에지가 3번인지 확인한다.
4. 동작 중 리셋을 넣고 초기화와 재시작을 검사한다.
5. 불일치는 LAB2_FAIL과 \$fatal로 즉시 중단한다.

tb_clock_divider.sv · 1-16행

sim/tb_clock_divider.sv를 열고 해당 구간을 직접 입력한다.

```
clock_divider.v U x lab2_clock_divider.v S, U x tb_clock_divider.sv U x input_frontend.v 1, U x
sim > tb_clock_divider.sv > ...
1 | timescale 1ns/1ps
2 module tb_clock_divider;
3 reg clk=0, rst=1;
4 integer checks=0;
5 always #5 clk=~clk;
6 task step; begin @(posedge clk); #1; end endtask
7 task check(input condition, input [8*100-1:0] description);
8 begin
9     checks=checks+1;
10    if (condition != 1'b1) begin
11        $display("LAB2_FAIL %0s time=%0t",description,$time);
12        $fatal(1,"check failed");
13    end
14 end
15 endtask
16 task finish;
```

tb_clock_divider.sv · 17-32행

sim/tb_clock_divider.sv를 열고 해당 구간을 직접 입력한다.

```
clock_divider.v U × lab2_clock_divider.v S, U × tb_clock_divider.sv U × input_frontend.v 1, U ×
sim > tb_clock_divider.sv >...
17 begin
18     $display("LAB2_PASS clock_divider checks=%0d",checks);
19     $finish;
20 end
21 endtask
22 initial begin $dumpfile("wave.vcd"); $dumpvars(0,tb_clock_divider); end
23 initial begin #100000; $fatal(1,"watchdog timeout"); end
24 wire divided, tick, div2, tick2;
25 clock_divider #(.DIVISOR(10)) dut(clk,rst,divided,tick);
26 clock_divider #(.DIVISOR(2)) minimum(clk,rst,div2,tick2);
27 integer i, pulses=0;
28 always @(posedge clk) if (!rst && tick) pulses = pulses+1;
29 initial begin
30     step; check([divided,tick,div2,tick2]===4'b0000,"reset");
31     rst=0;
32     for(i=1;i<=30;i=i+1) begin
```

tb_clock_divider.sv · 33–48행

sim/tb_clock_divider.sv를 열고 해당 구간을 직접 입력한다.

```
33     step;
34     check(divided==(i%10>=5),"divide 10 duty and phase");
35     check(tick==(i%10==9),"tick before consuming edge");
36     check(div2==(i%2==1),"minimum divisor duty");
37     check(tick2==(i%2==1),"minimum tick");
38     end
39     check(pulses===3,"three consumed enables in 30 clocks");
40     repeat(7) step;
41     rst=1; #1; check(tick===0,"reset masks tick before edge");
42     step; check(divided===0,"reset mid period");
43     rst=0;
44     repeat(4) begin step; check(divided===0,"restart low half"); end
45     step; check(divided===1,"restart first transition");
46     finish;
47 end
48 endmodule
```

XDC도 직접 입력

constraints/lab2_clock_divider.xdc에 다음 화면 전체를 작성한다.

get_ports 이름을 lab2_clock_divider.v의 포트와 대조한다.

Icarus는 XDC를 사용하지 않는다. PASS는 분주 코어의 기능 검사 결과다.
핀·보드 입출력·타이밍은 Vivado와 장치에서 별도로 확인한다.

lab2_clock_divider.xdc · 1-16행

constraints/lab2_clock_divider.xdc를 열고 해당 구간을 직접 입력한다.

```
1  # Combo II-DLD S75 / xc7s75fgga484-1 / main clock 1 kHz.
2  # Pin source and array polarity: evidence/board-pin-provenance.json.
3  set_property PACKAGE_PIN B6 [get_ports {clk}]
4  set_property IOSTANDARD LVCMOS33 [get_ports {clk}]
5  set_property PACKAGE_PIN K4 [get_ports {rst}]
6  set_property IOSTANDARD LVCMOS33 [get_ports {rst}]
7  set_property PACKAGE_PIN N8 [get_ports {button}]
8  set_property IOSTANDARD LVCMOS33 [get_ports {button}]
9  set_property PACKAGE_PIN U4 [get_ports {sw[0]}]
10 set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]
11 set_property PACKAGE_PIN V4 [get_ports {sw[1]}]
12 set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]
13 set_property PACKAGE_PIN W1 [get_ports {sw[2]}]
14 set_property IOSTANDARD LVCMOS33 [get_ports {sw[2]}]
15 set_property PACKAGE_PIN W4 [get_ports {sw[3]}]
16 set_property IOSTANDARD LVCMOS33 [get_ports {sw[3]}]
```

lab2_clock_divider.xdc · 17-32행

constraints/lab2_clock_divider.xdc를 열고 해당 구간을 직접 입력한다.

```
17 set_property PACKAGE_PIN T1 [get_ports {sw[4]}]
18 set_property IOSTANDARD LVCMOS33 [get_ports {sw[4]}]
19 set_property PACKAGE_PIN U2 [get_ports {sw[5]}]
20 set_property IOSTANDARD LVCMOS33 [get_ports {sw[5]}]
21 set_property PACKAGE_PIN W3 [get_ports {sw[6]}]
22 set_property IOSTANDARD LVCMOS33 [get_ports {sw[6]}]
23 set_property PACKAGE_PIN Y1 [get_ports {sw[7]}]
24 set_property IOSTANDARD LVCMOS33 [get_ports {sw[7]}]
25 set_property PACKAGE_PIN N5 [get_ports {led[0]}]
26 set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
27 set_property PACKAGE_PIN M1 [get_ports {led[1]}]
28 set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
29 set_property PACKAGE_PIN M3 [get_ports {led[2]}]
30 set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
31 set_property PACKAGE_PIN M7 [get_ports {led[3]}]
32 set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
```

lab2_clock_divider.xdc · 33-42행

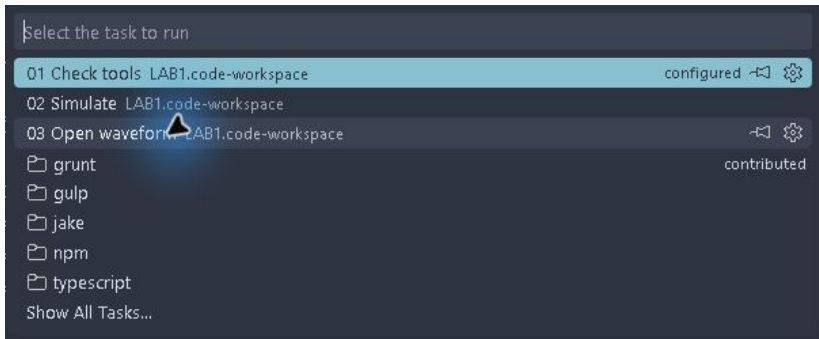
constraints/lab2_clock_divider.xdc를 열고 해당 구간을 직접 입력한다.

```
33 set_property PACKAGE_PIN N7 [get_ports {led[4]}]
34 set_property IOSTANDARD LVCMOS33 [get_ports {led[4]}]
35 set_property PACKAGE_PIN M2 [get_ports {led[5]}]
36 set_property IOSTANDARD LVCMOS33 [get_ports {led[5]}]
37 set_property PACKAGE_PIN M4 [get_ports {led[6]}]
38 set_property IOSTANDARD LVCMOS33 [get_ports {led[6]}]
39 set_property PACKAGE_PIN L4 [get_ports {led[7]}]
40 set_property IOSTANDARD LVCMOS33 [get_ports {led[7]}]
41 create_clock -name trainer_1khz -period 1000000.000 [get_ports clk]
42 set_false_path -from [get_ports {rst button sw[*]}]
```

VS Code에서 실행

1. File → Save All로 모든 파일을 저장한다.
2. Terminal → Run Task... 또는 Ctrl+Shift+P → Tasks: Run Task.
3. 01 Check tools로 설치 확인 후 02 Simulate를 선택한다.
4. Ctrl+Shift+B도 이 템플릿의 기본 작업인 02 Simulate를 실행한다.

작업 선택



번호 01은 도구 확인, 02는 컴파일·테스트벤치 실행, 03은 파형 파일 열기다.

Python 실행 경로 오류

WindowsApps·9009 오류가 나오면 실행 별칭이 실제 Python을 가리키는지 확인한다.

LAB1.code-workspace를 열고 세 작업의 windows → command에 자신의 python.exe 경로를 지정한다. JSON 경로에는 /를 쓰면 편리하다.

다른 사람의 사용자 이름·설치 경로를 그대로 복사하지 않는다. 수정한 항목을 실험 전 레포트에 적는다.

129개 검사 통과

```
VCD info: dumpfile wave.vcd opened for output.  
LAB2_PASS clock_divider checks=129
```

LAB2_PASS clock_divider checks=129. 바로 다음 \$finish 줄의 436000 ps
는 436 ns다. 같은 폴더에서 실행한 최신 로그인지 확인한다.

편집기 경고와 실행 결과

편집기의 정적 분석과 Icarus 실행은 서로 다른 검사다. 캡처에는 사용하지 않는 입출력 등에 대한 경고가 남아 있다.

unknown module 오류는 파일명.module 이름·저장 여부를 먼저 확인한다. 여러 파일의 모듈 탐색이 필요하다면 slang의 소스 검색 경로를 지정하고 언어 서버를 재시작한다.

PASS는 모든 가능한 입력을 증명한 결과가 아니다. 이번 TB의 129개 비교와 종료 시각을 확인한 결과다.

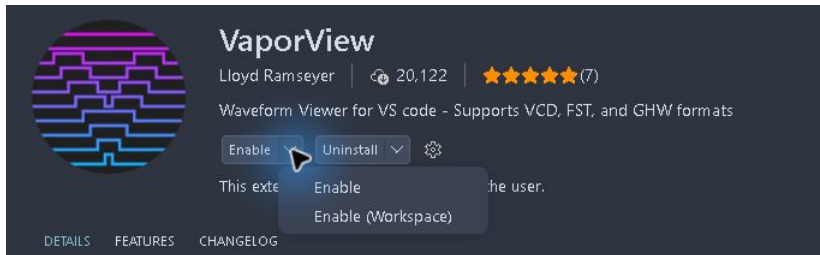
여러 RTL 파일의 편집기 설정



```
.slang > {} server.json > ...
1  {}
2  "flags": "-y src",
3  "index": [
4    {
5      "dirs": [
6        "src",
7        "sim"
8      ],
9      "excludeDirs": [
10       "build"
11     ]
12   }
13 ],
14 "inlayHints": {
15   "orderedInstanceNames": false
16 }
17 {}
```

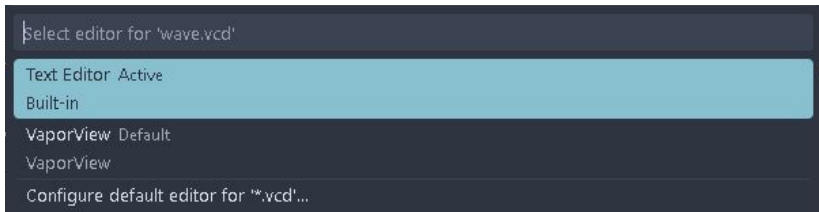
v2.0.1에는 .slang/server.json이 포함되어 있다. 이 화면은 이전 템플릿에서 설정을 추가한 예다. 현재 파일에서 src 탐색 설정을 확인한다. src를 모듈 검색 경로로 지정한다. 저장 후 명령 팔레트의 Developer: Reload Window로 다시 연다.

VaporView가 꺼져 있다면



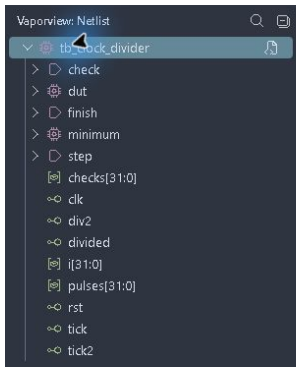
Extensions → @installed vaporview → Enable 옆 화살표 → Enable (Workspace).

텍스트 VCD를 파형으로 전환



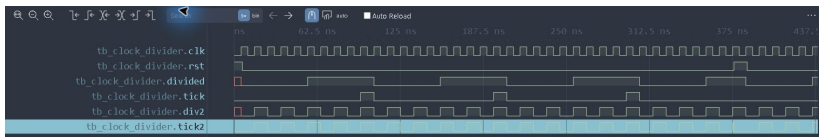
build/sim/wave.vcd를 연다. 텍스트라면 탭 우클릭 → Reopen Editor With... → VaporView.

관찰할 신호 추가



Netlist View → tb_clock_divider를 펼친다. clk·rst·divided·tick·div2·tick2를 차례로 더블클릭한다.

전체 파형과 시간축



왼쪽 위 Zoom Fit으로 전체 구간을 맞춘다. 이름 열 경계를 넓히고, 하단 Time Units → ns를 선택한다.

10분주 파형 해설

rst가 해제된 뒤 divided는 55 ns에서 처음 1이 된다. 이후 105 ns에서 0, 155 ns에서 다시 1이다.

High 50 ns와 Low 50 ns를 합친 주기 100 ns는 입력 주기 10 ns의 10배다.

tick은 95-105 ns, 195-205 ns, 295-305 ns에 1이다. 다음 상승 에지에서 enable로 소비되므로 30클록 동안 3회가 된다.

리셋 파형 해설

376 ns에 $\text{rst}=1$. tick은 리셋 조건에 의해 즉시 0으로 마스킹된다.

385 ns 상승 에지에서 count와 divided가 초기화된다. $\text{rst}=0$ 으로 돌아온 후 새 주기를 센다.

435 ns에서 divided가 다시 1이 되고, TB는 436 ns에 마지막 검사 후 종료한다. 처음 0-5 ns의 X는 첫 동기 리셋 에지 전의 미정 상태다.

수정 실험: 한 클록의 차이

1. 정상 로그와 wave.vcd를 별도 보관한다.
2. clock_divider.v에서 상승 조건 $\text{DIVISOR}/2 - 1$ 을 $\text{DIVISOR}/2$ 로 바꾼다.
3. TB는 그대로 두고 저장 → 02 Simulate.
4. minimum divisor duty 검사가 16 ns에 실패하는 이유를 계산한다.
5. 조건을 원복하고 다시 실행해 129개 검사 통과를 확인한다.

실험 전 레포트

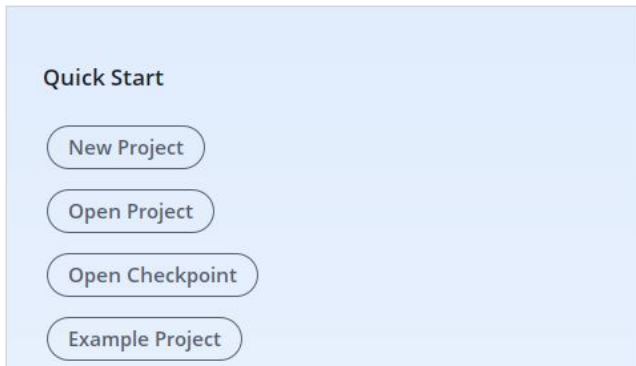
1. count.divided.tick의 예상 상태와 분주 주기를 계산한다.
2. RTL.TB.simulation.json.XDC의 역할을 설명한다.
3. 자신의 정상/변경/원복 로그를 표로 비교한다.
4. 전체 파형과 듀티.tick.리셋 확대 화면을 넣고 해석한다.
5. 1 kHz 보드 설정, 각 LED 예상 동작과 측정 방법을 정리한다.

Vivado에서 이어서 진행

VS Code의 정상·변경·원복 검사와 실험 전 레포트를 마친 뒤 Vivado를 연다.

지금부터도 같은 학생 폴더의 소스를 사용한다. Design top은 lab2_clock_divider, Simulation top은 tb_clock_divider다.

새 프로젝트



Quick Start → New Project. 이름은 lab2_clock_divider로 정한다.

프로젝트 이름과 위치

Project name: ×

Project location: × ...

☐ Create project subdirectory

Project will be created at: C:/Users/UOS/Documents/ece2_26_2/tmp/lab2-students-0913/lab2_02_clock_divider/vivado

Project location은 자신의 lab2_02_clock_divider/vivado. Create project subdirectory를 해제하고 Next.

RTL Project 선택

- ☒ **RTL Project**
You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL
- ☒ **Do not specify sources at this time**
- ☐ Project is an extensible Vitis platform

RTL Project와 Do not specify sources at this time을 선택 → Next.

정확한 부품 선택

Parts Boards

Q

Q: xc7s75fgga484-1 (3 matches)

Part	I/O Pin Count	Available IOBs	LUT Elements	FlipFI
xc7s75fgga484-1	484	338	48000	9600
xc7s75fgga484-1IL	484	338	48000	9600
xc7s75fgga484-1Q	484	338	48000	9600

Parts에서 xc7s75fgga484-1을 검색하고 해당 행을 선택 → Next. 패키지 fgga484와 속도 등급 -1도 확인한다.

생성 전 확인

New Project Summary

- A new RTL project named 'lab2_clock_divider' will be created.
- The default part and product family for the new project:
Default Part: xc7s75fgga484-1
Family: Spartan-7
Package: fgga484
Speed Grade: -1

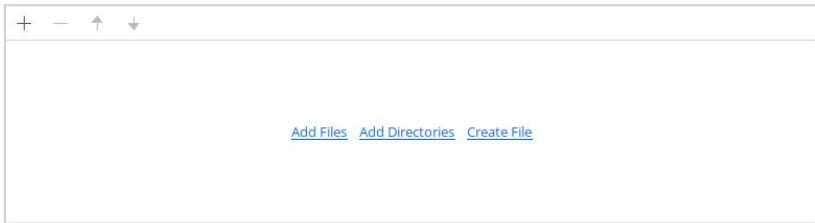
프로젝트 이름·위치·부품이 맞으면 Finish. 보드 top과 테스트벤치는 다음 단계에서 각각 등록한다.

설계 소스 추가

- ☐ Add or create constraints
- ☒ Add or create design sources
- ☐ Add or create simulation sources

Flow Navigator → Add Sources → Add or create design sources → Next.

RTL 파일 찾기



Add Files → 학생 폴더의 src를 연다. 앞에서 이름을 바꾸고 작성한 RTL 파일을 고른다.

RTL 세 파일 선택

Look in

src

- clock_divider.v
- input_frontend.v
- lab2_clock_divider.v

clock_divider.v·input_frontend.v·lab2_clock_divider.v를 선택 → OK.
이 폴더에 세 파일만 있으면 목록 클릭 후 Ctrl+A로 선택할 수 있다.

원본 소스 연결

	Index	Name	Library	Version	Location
●	1	clock_divider.v	xil_defaultlib	N/A	C:/Users/UOS/Documents/ece2_26_2_2/tmp/l
●	2	input_frontend.v	xil_defaultlib	N/A	C:/Users/UOS/Documents/ece2_26_2_2/tmp/l
●	3	lab2_clock_divider.v	xil_defaultlib	N/A	C:/Users/UOS/Documents/ece2_26_2_2/tmp/l

- ☒ Scan and add RTL include files into project
- ☐ Copy sources into project
- ☐ Add sources from subdirectories

목록의 세 파일을 확인한다. Copy sources into project를 해제 → Finish. VS Code에서 저장한 원본을 계속 참조한다.

시뮬레이션 소스 추가

- ☐ Add or create constraints
- ☐ Add or create design sources
- ☒ Add or create simulation sources

Add Sources → Add or create simulation sources → Next → Add Files →
sim/tb_clock_divider.sv.

테스트벤치 등록 확인

Specify simulation set: sim_1

+	-	↑	↓		
	Index	Name	Library	Version	Location
●	1	tb_clock_divider.sv	xil_defaultlib	N/A	C:/Users/UOS/Documents/ece2_26_2_2/tmp/la
< >					

- ☒ Scan and add RTL include files into project
- ☐ Copy sources into project
- ☐ Add sources from subdirectories
- ☒ Include all design sources for simulation

sim_1과 Include all design sources for simulation을 유지한다. Copy sources into project 해제 → Finish.

제약 소스 추가

- ☒ Add or create constraints
- ☐ Add or create design sources
- ☐ Add or create simulation sources

Add Sources → Add or create constraints → Next → Add Files → constraints/lab2_clock_divider.xdc.

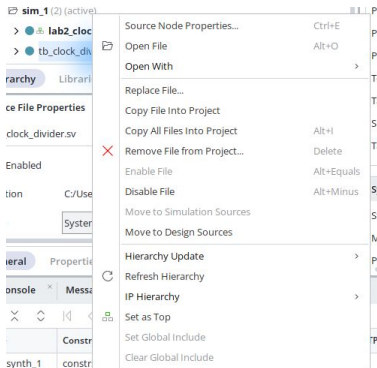
XDC 등록 확인

+ - ↑ ↓	
Constraint File	Location
lab2_clock_divider.xdc	C:\Users\UOS\Documents\ece2_26_2_2\tmp\lab2-students-0913\lab2_02_clock_divider\src\..\sim\..\cons

☐ Copy constraints files into project

Copy constraints files into project 해제 → Finish. XDC의 수정도 학생 폴더 원본에 저장한다.

TB를 최상위로 지정



Sources → Simulation Sources → sim_1을 펼친다. tb_clock_divider 우클릭 → Set as Top.

두 Top을 구분

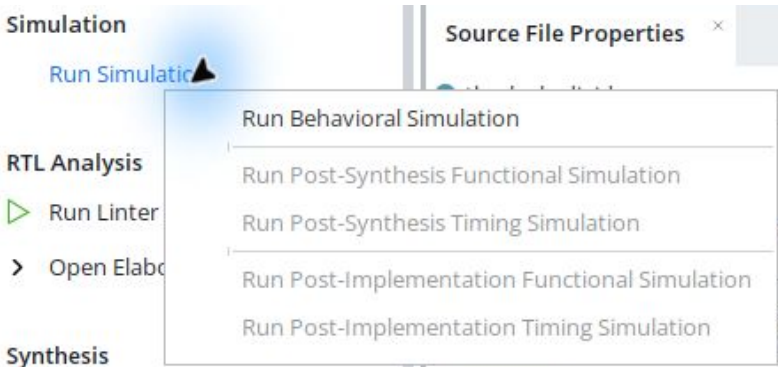


The screenshot displays the Vivado IDE interface. On the left, the 'Simulation Sources' pane shows a project named 'sim_1' (active) containing two sources: 'tb_clock_divider (tb_clock_divider.sv) (2)' and 'lab2_clock_divider (lab2_clock_divider.v) (5)'. The 'tb_clock_divider' source is highlighted. Below this pane are tabs for 'Hierarchy', 'Libraries', and 'Compile Order'. On the right, the 'Project Summary' pane lists the following details:

Project name	lab2_clock_divider
Project location	C:/Users/UOS/Documents/ece2_4
Product family	Spartan-7
Project part	xc7s75fpga484-1
Top module name	lab2_clock_divider

Simulation Sources에서 tb_clock_divider가 굵게 표시된다. Project Summary의 설계 Top은 lab2_clock_divider여야 한다.

동작 시뮬레이션 실행



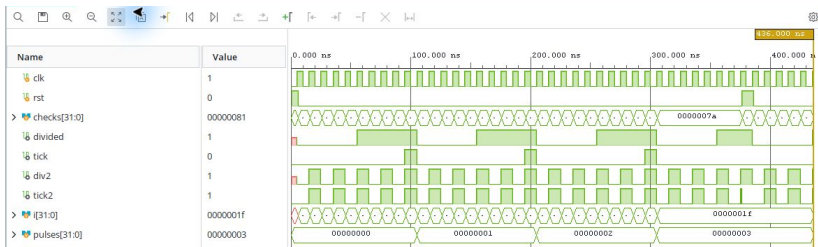
Flow Navigator → Run Simulation → Run Behavioral Simulation.
컴파일과 elaboration이 끝날 때까지 기다린다.

동일한 129개 검사 통과

```
# }  
# run 1000ns  
LAB2_PASS clock_divider checks=129  
$finish called at time : 436 ns : F
```

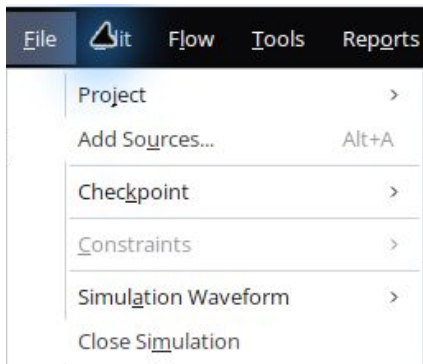
Tcl Console에서 LAB2_PASS clock_divider checks=129와 436 ns 종료를 확인한다. PASS가 위로 숨으면 콘솔을 조금 위로 스크롤한다.

Vivado 전체 파형



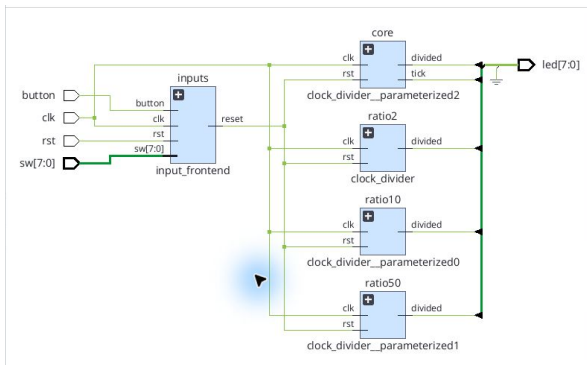
Untitled 1 파형 탭을 더블클릭해 넓힌 뒤 Zoom Fit을 누른다. divided 주기 100 ns, tick 폭 10 ns와 동작 중 리셋을 VS Code 파형과 대조한다.

시뮬레이션 종료



File → Close Simulation → OK. 캡처와 레포트용 로그를 먼저 보관한다.

RTL 회로 연결 확인



RTL Analysis → Open Elaborated Design → 안내 창 OK. 주 클럭이 각 분주 회로에 연결되고 출력이 LED로 이어지는지 확인한다.

핀 표 열기



Window → I/O Ports. 아래에 열린 I/O Ports 탭을 더블클릭하면 표가 넓어진다.

LED와 스위치 핀

Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std
▼ All ports (19)						
▼ led (8)	OUT			✓	35	LVC MOS33* ▼
led[7]	OUT		L4	✓	35	LVC MOS33* ▼
led[6]	OUT		M4	✓	35	LVC MOS33* ▼
led[5]	OUT		M2	✓	35	LVC MOS33* ▼
led[4]	OUT		N7	✓	35	LVC MOS33* ▼
led[3]	OUT		M7	✓	35	LVC MOS33* ▼
led[2]	OUT		M3	✓	35	LVC MOS33* ▼
led[1]	OUT		M1	✓	35	LVC MOS33* ▼
led[0]	OUT		N5	✓	35	LVC MOS33* ▼
▼ sw (8)	IN			✓	34	LVC MOS33* ▼
sw[7]	IN		Y1	✓	34	LVC MOS33* ▼
sw[6]	IN		W3	✓	34	LVC MOS33* ▼
sw[5]	IN		U2	✓	34	LVC MOS33* ▼
sw[4]	IN		T1	✓	34	LVC MOS33* ▼
sw[3]	IN		W4	✓	34	LVC MOS33* ▼
sw[2]	IN		W1	✓	34	LVC MOS33* ▼
sw[1]	IN		V4	✓	34	LVC MOS33* ▼
sw[0]	IN		U4	✓	34	LVC MOS33* ▼

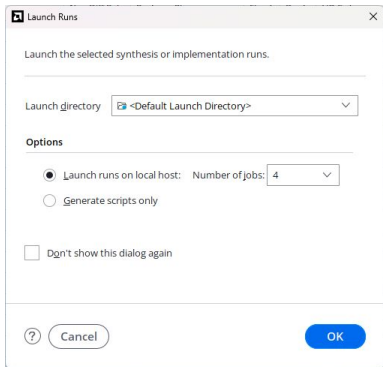
All ports 우클릭 → Expand Selection. 각 비트의 Package Pin·I/O Std를 XDC와 대조한다.

클록과 버튼 핀

Scalar ports (3)								
button	IN		N8	▽	✓	35	LVC MOS33*	▽
clk	IN		B6	▽	✓	36	LVC MOS33*	▽
rst	IN		K4	▽	✓	35	LVC MOS33*	▽

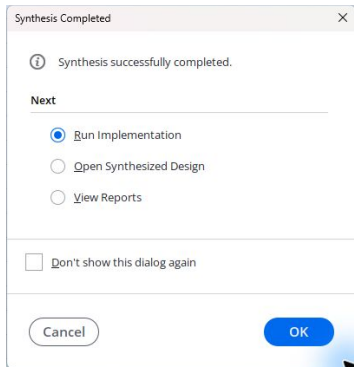
Scalar ports도 펼친다. clk=B6, rst=K4, button=N8, 모두 LVC MOS33인지 확인한다.

합성 실행



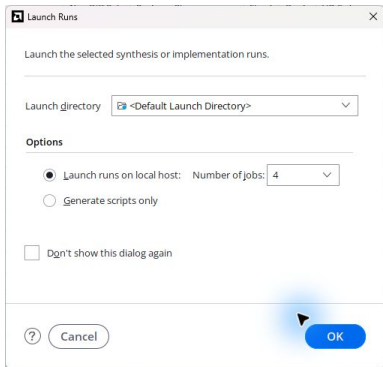
Flow Navigator → Run Synthesis. Launch runs on local host와 jobs를 확인 → OK. 이 화면은 jobs=4로 실행했다.

합성 완료 후 구현



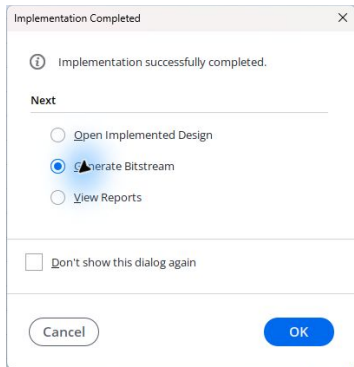
Synthesis successfully completed를 확인한다. Run Implementation 선택
→ OK.

구현 실행



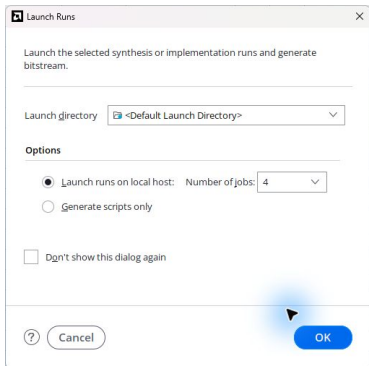
Launch runs on local host와 jobs를 확인 → OK. 배치배선 중에는 소스와 XDC를 수정하지 않는다.

구현 완료 후 bit 생성



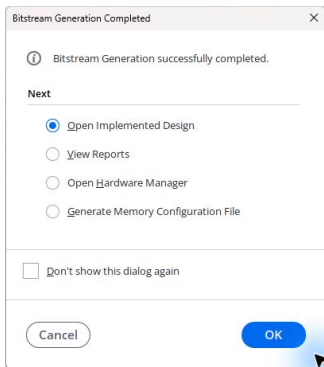
Implementation successfully completed를 확인한다. Generate Bitstream 선택 → OK.

비트스트림 실행



Launch runs on local host와 jobs를 확인 → OK. 오른쪽 위 Running write_bitstream이 완료될 때까지 기다린다.

비트스트림 생성 완료



Bitstream Generation successfully completed를 확인한다. Open Implemented Design 선택 → OK. bit 생성 완료와 실제 장치 기록은 서로 다른 단계다.

구현 타이밍 결과

Design Timing Summary

Setup		Hold	
Worst Negative Slack (WNS)	999996.312 ns	Worst Hold Slack (WHS)	0.122 ns
Total Negative Slack (TNS)	0.000 ns	Total Hold Slack (THS)	0.000 ns
Number of Failing Endpoints	0	Number of Failing Endpoints	0
Total Number of Endpoints	51	Total Number of Endpoints	51

Elaborated Design을 닫을지 묻는 창은 Yes. 아래 Timing → Design Timing Summary에서 Setup·Hold의 slack과 Failing Endpoints를 확인한다.

생성 파일과 검증 범위

생성 파일:

`vivado/lab2_clock_divider.runs/impl_1/lab2_clock_divider.bit`.

이 교안의 실제 실행은 Vivado 2026.1에서 동일 학생 폴더의 시뮬레이션·합성·구현·bit 생성까지 확인했다.

다음 장치 기록 절차는 학생이 실습 장비에서 수행한다. 이 교안의 bit 생성 화면은 보드 동작을 촬영한 결과가 아니다.

남은 경고도 레포트에 기록

구현 보고서의 $WNS=999996.312\text{ ns}$, $WHS=0.122\text{ ns}$. 지정한 내부 타이밍 제약은 만족했다. 입력 주기가 $1,000,000\text{ ns}$ 이므로 WNS가 크게 나온다.

TIMING-18: 입출력 지연 제약이 빠져 있다. LED 관찰용 출력에는 외부 동기 수신기의 타이밍 조건을 정의하지 않았다. 내부 타이밍 통과를 모든 외부 입출력 타이밍 보장으로 해석하지 않는다.

CFGBVS-1: 설정 बैं크의 전압 속성이 지정되지 않았다. 보드 회로도·설정 전압을 확인한 뒤 값을 정하며, 경고를 없애려고 임의의 전압을 입력하지 않는다.

장치에 기록

1. 보드 전원·JTAG 연결·주 클록 1 kHz를 확인한다.
2. Open Hardware Manager → Open Target → Auto Connect.
3. 장치 우클릭 → Program Device → 방금 만든 lab2_clock_divider.bit → Program.
4. K4 초기화 후 LED[3]의 1초 주기와 다른 출력의 분주비를 확인한다.
5. 빠른 신호는 눈으로 주파수를 단정하지 말고 측정 장비나 파형으로 비교한다.

실험 후 레포트

1. Vivado의 동일 TB PASS·파형을 VS Code 결과와 비교한다.
2. 부품·핀·클록 제약·합성·구현·bit 파일과 남은 경고를 기록한다.
3. 실제 장치 기록 화면, 보드 전체와 LED 사진을 넣는다.
4. 초기화와 1 Hz 출력 변화를 함께 보여 주는 영상을 촬영한다.
5. 소스·실험 전후 레포트·사진·영상 링크를 GitHub README에서 연결한다.